PlantsFarm

Enrico Michelini, Diego Bruschi, Francesco Fusillo

Indice

1 **Analisi**

1.1 Descrizione e requisiti

1.2 Modello del Dominio

2 **Design**

2.1 Architettura

2.2 Design dettagliato

2.2.1 Enrico Michelini

2.2.2 Diego Bruschi

2.2.3 Francesco Fusillo

3 **Sviluppo**

3.1 Testing automatizzato

3.1.1 Enrico Michelini

3.1.2 Diego Bruschi

3.1.3 Francesco Fusillo

3.2 Note di sviluppo

3.2.1 Enrico Michelini

3.2.2 Diego Bruschi

3.2.3 Francesco Fusillo

4 **Commenti finali**

4.1 Autovalutazione e lavori futuri

4.1.1 Enrico Michelini

4.1.2 Diego Bruschi

4.1.3 Francesco Fusillo

Appendice A
Guida utente

Analisi

1.1 Descrizione e Requisiti

Il software implementa una simulazione gestionale agricola in cui l'utente assume il controllo di un avatar all'interno di un'area di gioco predefinita. L'ambiente è strutturato per offrire la possibilità della coltivazione in sei distinti appezzamenti di terreno organizzati in matrici 5x5 con 8 zolle coltivabili, quella centrale destinata alle piante ornamentali mentre intorno destinate alle piante edibili.

All'inizio della partita, il giocatore ha accesso a una singola tipologia di coltura, la pianta di carote. Per sbloccare le restanti specie vegetali, catalogate in un'apposita enciclopedia, è necessario avviare un ciclo produttivo che prevede la coltivazione, la raccolta e la successiva vendita dei frutti presso il negozio. Questa transazione genera valuta di gioco, indispensabile per acquisire semi di piante via via più rare. Ogni specie vegetale è caratterizzata da attributi unici, quali un prezzo di vendita e un tempo di maturazione specifico.

Ogni pianta attraversa diversi stadi evolutivi prima di raggiungere la maturazione completa, momento in cui diviene possibile il raccolto. Durante il ciclo vitale, il sistema richiede l'intervento attivo del giocatore per operazioni di mantenimento, ovvero l'irrigazione, condizione necessaria per l'avanzamento della crescita. Una caratteristica peculiare del modello di simulazione adottato riguarda la persistenza delle colture: una volta raccolti i frutti, la pianta non viene rimossa né ritorna allo stadio iniziale, ma regredisce a uno stadio vegetativo intermedio, permettendo così nuovi cicli di fioritura e fruttificazione senza la necessità di una nuova semina.

Le sfide per il player saranno due: la prima sarà quella di gestire adeguatamente le sue finanze per investire in nuove piante, tra le quali sono presenti quelle ornamentali, che lo aiuteranno a progredire fornendogli aiuti relativi alla velocità di crescita delle colture che circondano la relativa pianta ornamentale o alla quantità di prodotto fornita al momento della raccolta. La seconda sfida consisterà nel gestire i punti esperienza guadagnati durante il gameplay tramite l'utilizzo dei rispettivi utensili o tramite raccolta delle ampolle di esperienza che vengono generate casualmente sulla mappa per aumentare il livello degli strumenti ed essere in grado di interagire con piante di rarità superiore.

Requisiti funzionali

- **Gestione dell'Avatar:** Il sistema deve permettere al giocatore di controllare un avatar che può muoversi liberamente all'interno dell'area di gioco, rispettando le collisioni con oggetti solidi.
- **Interazione Ambientale:** Il sistema deve fornire comandi per interagire con il terreno, permettendo azioni quali: seminare, innaffiare e raccogliere.
- **Ciclo Vitale delle Piante:** Il sistema deve simulare la crescita delle piante attraverso stadi successivi. La crescita deve essere condizionata dal trascorrere del tempo e dallo stato di irrigazione del terreno.
- **Meccanica di Raccolta e Rigenerazione:** Il sistema deve garantire che, dopo il raccolto, le piante non vengano rimosse ma regrediscono a uno stadio precedente, permettendo cicli di raccolta multipli.

- **Gestione Economica:** Il sistema deve includere un negozio virtuale che permetta al giocatore di vendere i frutti raccolti per ottenere monete e acquistare nuovi semi.
- **Gestione Storage:** Il sistema deve fornire un magazzino per accumulare i prodotti raccolti prima della vendita.
- **Progressione e Livelli:** Il sistema deve gestire un punteggio di esperienza per i singoli strumenti, sbloccando la capacità di interagire con piante di rarità superiore.
- **Piante Ornamentali:** Il sistema deve supportare piante ornamentali che applicano effetti passivi (aumento velocità di crescita o quantità del raccolto) alle colture vicine.
- **Enciclopedia:** Il sistema deve fornire un'interfaccia consultabile che mostri lo stato di sblocco delle varie specie vegetali e le loro caratteristiche.
- **Animazioni:** Il sistema deve visualizzare feedback visivi per il movimento del giocatore e per l'esecuzione delle azioni.

Requisiti non funzionali

- **Persistenza dei Dati:** Il sistema deve essere in grado di salvare lo stato del gioco su file locale e di ricaricarlo al riavvio dell'applicazione.
- **Usabilità:** L'interfaccia grafica deve essere intuitiva, con pulsanti e menu chiaramente identificabili.
- **Estensibilità:** L'architettura software deve permettere l'aggiunta di nuove tipologie di piante o comportamenti senza dover ristrutturare l'intero sistema.
- **Affidabilità:** Il sistema deve gestire eccezioni legate al caricamento dei file senza causare il blocco dell'applicazione.

1.2 Modello del Dominio

La struttura di PlantsFarm si basa su una netta separazione tra la logica di stato, l'ambiente fisico e il ciclo economico:

I dati statici delle piante, il bilancio della valuta e i frutti raccolti sono centralizzati nel GameState, che garantisce che le informazioni sulle singole istanze siano sempre sincronizzate e disponibili per gli altri modelli.

L'entità Plant rappresenta l'elemento centrale del gioco, ogni pianta trae le proprie caratteristiche e i propri comportamenti dal PlantType, che ne definisce gli attributi.

Le piante risiedono fisicamente nelle unità Soil, ovvero le zolle coltivabili della mappa, le quali sono raggruppate e coordinate dal modello Garden.

Il Player interagisce con il mondo di gioco agendo direttamente sulle unità Soil tramite gli strumenti gestiti nell'Inventory e attraverso queste interazioni, il giocatore può influenzare il ciclo vitale della pianta innaffiandola fino alla maturazione e la raccolta.

Una volta completato il raccolto, i frutti passano allo Storage e possono essere venduti nello Shop tramite il GameState. Questa operazione genera nuova valuta, permettendo l'acquisto di nuove piante.

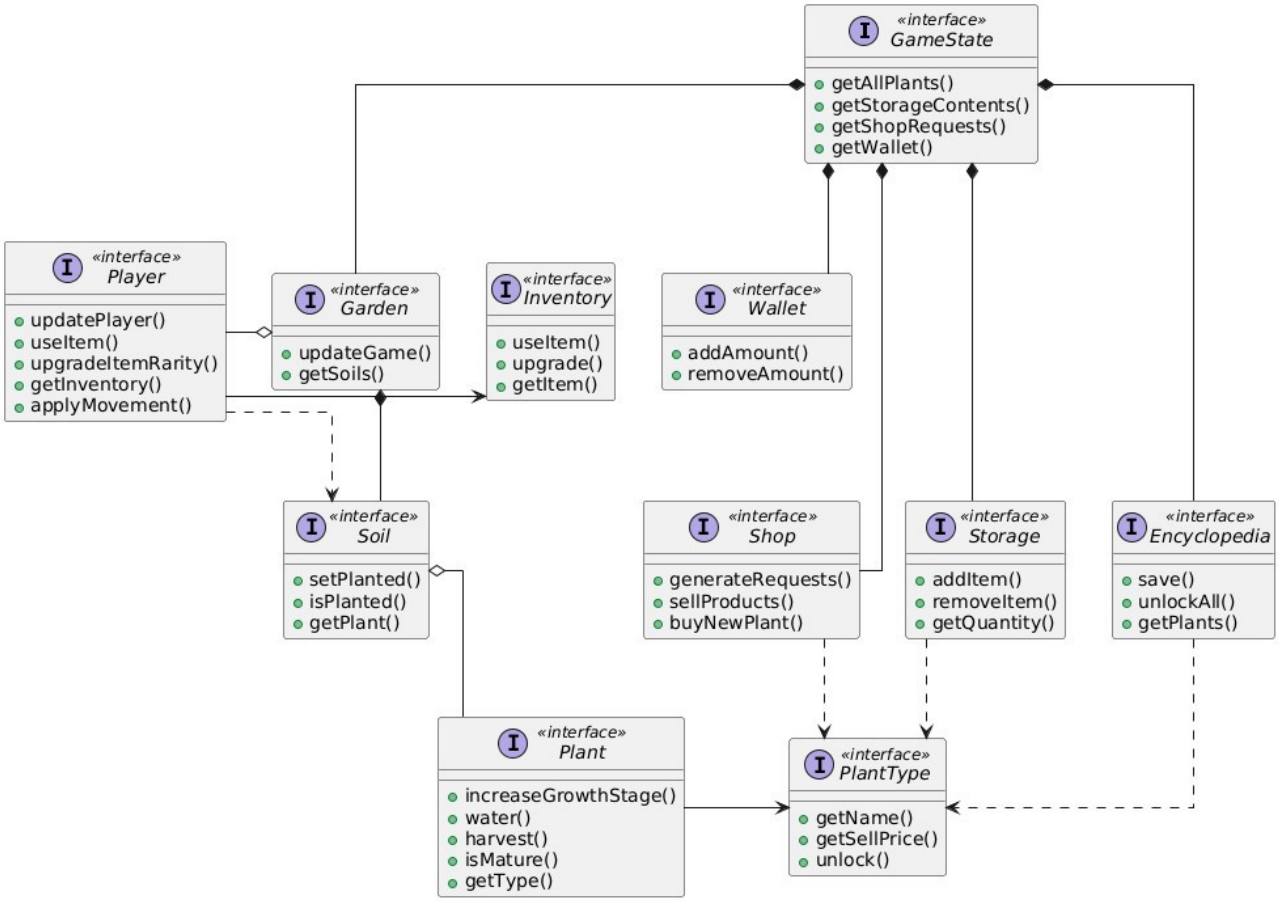

Figura 1: schema UML sul modello del dominio.

Design

2.1 Architettura

L'architettura del sistema PlantsFarm è stata progettata seguendo il pattern architetturale ModelView-Controller (MVC). Questa scelta è stata dettata dalla necessità di separare la logica di dominio (regole di crescita delle piante, economia, gestione inventario) dalla rappresentazione grafica. Tale separazione garantisce una maggiore manutenibilità del codice e la possibilità di modificare o sostituire l'interfaccia grafica senza impattare sulla logica interna del gioco.

L'applicazione è strutturata in tre macro-componenti:

1. Il Model

Il punto di ingresso del modello dei dati statici è rappresentato dalla classe GameState. Questo componente incapsula: il contenuto del magazzino, lo stato del portafoglio virtuale, l'enciclopedia delle piante e la logica della compravendita delle piante. Il modello contiene regole e dati, da manipolare o estrarre in base alle azioni richieste. Il punto di ingresso dei dati dinamici invece avviene attraverso vari modelli indipendenti, ognuno creato per risolvere una problematica differente: in particolare i modelli principali sono quelli di gestione delle piante, della mappa logica, dei buff, del player e dell'inventario.

2. La View

La View ha come punto d'ingresso la classe MainScreen, che gestisce il contenitore principale dell'applicazione e coordina i sottocomponenti visivi come ViewGamePanel (per la mappa) e MenuPanel (per l'interfaccia utente). La View ha due compiti principali:

Visualizzare lo stato attuale del gioco. La View interroga il Model tramite Controller per sapere cosa disegnare, ma senza eseguire calcoli logici.

Catturare le interazioni dell'utente come la pressione di tasti che informano il Controller sulle azioni da eseguire.

3. Il Controller

Il ruolo di Controller per la gestione dei dati statici è ricoperto principalmente dalla classe MainScreenController, che funge da punto di ingresso per la logica di gestione. Per rispettare il Principio di Singola Responsabilità, l'architettura prevede una distribuzione del controllo: il MainScreenController delega l'esecuzione di logiche complesse e specifiche a una serie di sotto-controller specializzati. Questa struttura gerarchica migliora la modularità e facilita la manutenzione del codice.

Il controller per i dati dinamici è centrallizzato all'interno di ControllerGamePanel, in quanto è responsabile dell'aggiornamento della ViewGamePanel e della comunicazione fra tutti gli altri controller.

2.2 Design Dettagliato

2.2.1 Enrico Michelini

Gestione centralizzata e coordinamento dei dati.

1. Problema

L'architettura del gioco richiede la gestione simultanea di diversi domini informativi tra loro indipendenti: la gestione economica, la persistenza dei raccolti, la progressione del giocatore e la logica della compravendita.

La sfida principale risiede nell'impostare una struttura che renda l'accesso a questi dati intuitivo per i vari Controller.

2. Soluzione

La soluzione adottata segue uno stile basato sul “Single Responsibility Principle”: ogni funzione principale è stata isolata in una classe specifica (Coin, Storage, Encyclopedia e Shop).

Per gestire i modelli è stato scelto di implementare una classe che funge da punto di ingresso, il GameState, usando il pattern Facade.

Quando un componente esterno deve consultare o modificare lo stato del gioco, non interagisce con i singoli modelli ma si rivolge esclusivamente al GameState, che agisce come un coordinatore, delegando il compito al modello richiesto.

Questa struttura semplifica il flusso di dati, viene inizializzato un unico oggetto GameState che contiene i dati principali del gioco; Quindi, passandolo come parametro è possibile svolgere tutte le operazioni necessarie alla gestione dei dati.

3. Schema UML

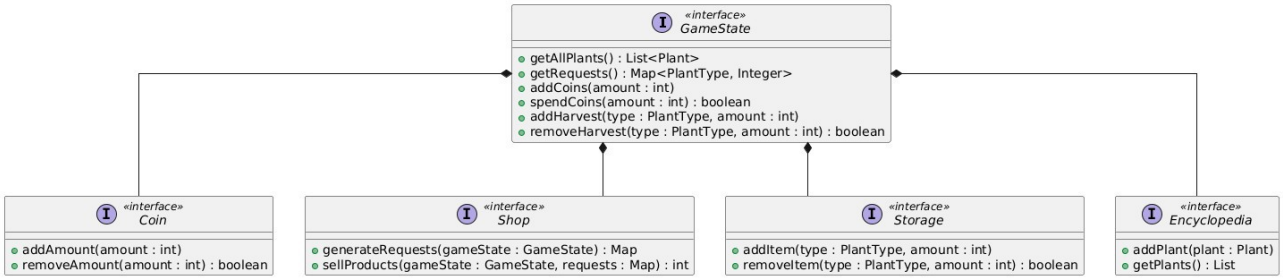

Figura 2: schema UML sulla gestione dei dati.

Rappresentazione visiva dei dati.

1. Problema

La gestione di interfacce multiple (Shop, Storage, Encyclopedia) richiede una standard stilistico e funzionale per evitare la duplicazione di codice e l'accoppiamento rigido tra logica e grafica.

2. Soluzione

Per garantire il principio di “Don’t repeat yourself”, sono state implementate delle classi Factory (ButtonFactory e WindowFactory) che si occupano della creazione generale dei componenti, così da semplificare la modifica di elementi stilistici di più classi in un solo punto.

La divisione tra logica e grafica è stata ottenuta delegando la gestione degli eventi ai Controller tramite l'uso di ActionListener; La View non implementa la logica dei modelli, ma espone dei metodi di aggancio.

Il Controller definisce il comando e lo attacca nella View; Questo approccio ha permesso di mantenere le classi grafiche facilmente sostituibili e modificabili senza impattare sul funzionamento del gioco.

Con il Command Pattern, la View non ha bisogno di conoscere l’interfaccia del Modello o del Controller, riceve un’istruzione da eseguire al click dell’utente. È stato valutato l’uso del Pattern Proxy, ma avrebbe invece costretto la View a interfacciarsi con un oggetto che ha la stessa forma del modello originale, mantenendo un legame più stretto tra le parti, per questo si è preferito l’uso del primo.

3. Schema UML

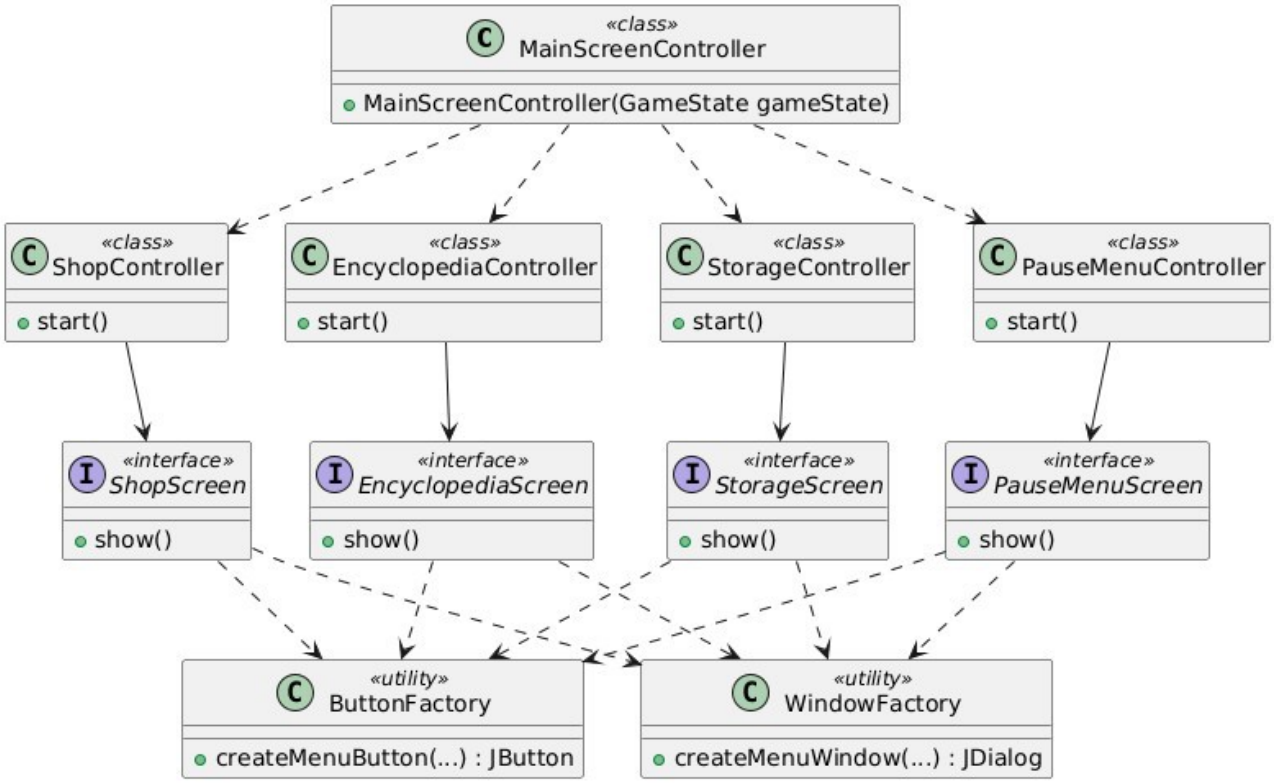

Figura 3: schema UML sulla rappresentazione dei dati del GameState.

Gestione tipologie di piante e il loro comportamento

1. Problema

È necessario trovare una struttura per distinguere tutte le tipologie di piante e il loro comportamento, ovvero dividerle in piante edibili e ornamentali.

2. Soluzione

È stato implementato il Pattern Strategy per separare la definizione della pianta dal suo comportamento logico. La classe PlantType funge da contesto e delega l'esecuzione delle operazioni economiche e di raccolta all'interfaccia PlantBehavior.

- EdibleBehavior: implementa la logica per le piante edibili, gestendo prezzi e calcolo casuale del raccolto.
- OrnamentalBehavior: implementa la logica per le piante ornamentali, gestendo bonus passivi.

I dati relativi a ogni pianta sono inseriti nella classe PlantRegistry che funge da punto di accesso unico per tutte le istanze di PlantType.

È stata valutata la possibilità di implementare la logica direttamente all'interno di un'Enum sfruttando il pattern Singleton, tuttavia, questa scelta è stata scartata per i seguenti motivi:

- Aggiungere un nuovo comportamento di una nuova tipologia di piante avrebbe reso il più difficile da leggere e comprendere, mentre con l'uso del Pattern Strategy sarebbe bastato creare una nuova classe che implementasse PlantBehavior.
- Separazione minore tra logica dei comportamenti e dati delle piante usando il pattern Singleton.
- Codice più sporco, con l'Enum ogni pianta conteneva tutti i comportamenti ma posti a valore nullo quelli indesiderati.

3. Schema UML

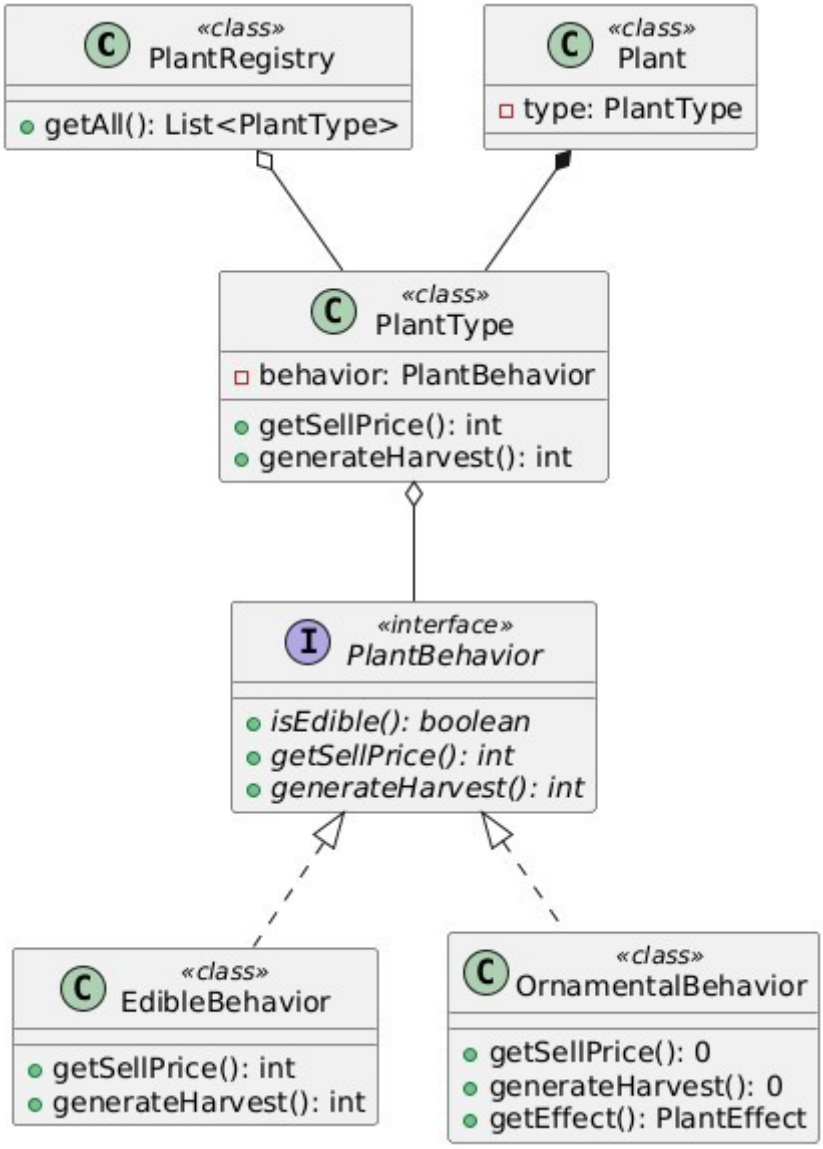

Figura 4: schema UML sulla rappresentazione dei dati e del comportamento delle piante.

Persistenza dei dati tra più sessioni di gioco.

1. Problema

È necessario salvare i dati all'esterno del file per poter progredire nel gioco con gli stessi salvataggi della partita precedente.

2. Soluzione

È stata implementata la classe DataMemory per scrivere i salvataggi in appositi file.txt all'esterno dell'applicazione; La gestione della scrittura e lettura dei file spetta al model che a inizio partita caricano i salvataggi dal file e durante la partita effettuano operazioni di salvataggio in modo da mantenere i progressi una volta chiuso il gioco.

3. Schema UML

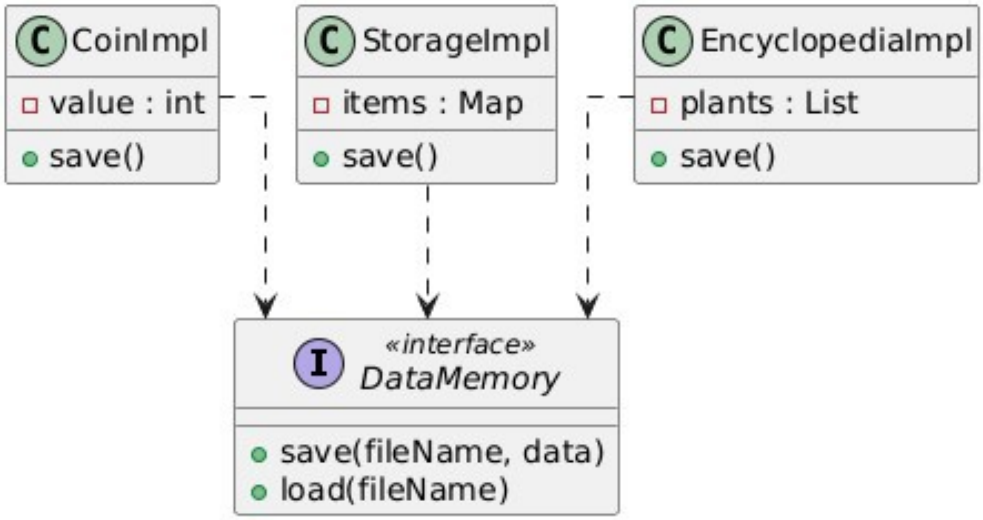

Figura 5: schema UML sulla memorizzazione dei dati.

Gestione delle interazioni spaziali e dei bonus passivi.

1. Problema

È necessario implementare un sistema che permetta alle piante di interagire tra loro a livello spaziale. Il problema consiste nel gestire i bonus passivi generati dalle piante ornamentali mature, che devono influenzare la velocità di crescita o la resa del raccolto delle piante edibili circostanti (appartenenti alla stessa matrice).

2. Soluzione

Per risolvere il problema della coordinazione spaziale, è stata introdotta la classe PlantArea. Questa entità rappresenta una porzione di 5x5 caselle del giardino e contenendo in una lista le zolle coltivabili presenti nell'area.

Ogni area identifica una zolla centrale specifica, se in questa posizione è presente una pianta ornamentale matura, l'area estrae il tipo di effetto e il relativo valore numerico tramite l'interfaccia OrnamentalBehavior.

Durante la fase di aggiornamento, PlantArea calcola i coefficienti di crescita (growthMultiplier) e di raccolta (harvestMultiplier) e li applica dinamicamente a tutte le altre piante presenti nell'area.

3. Schema UML

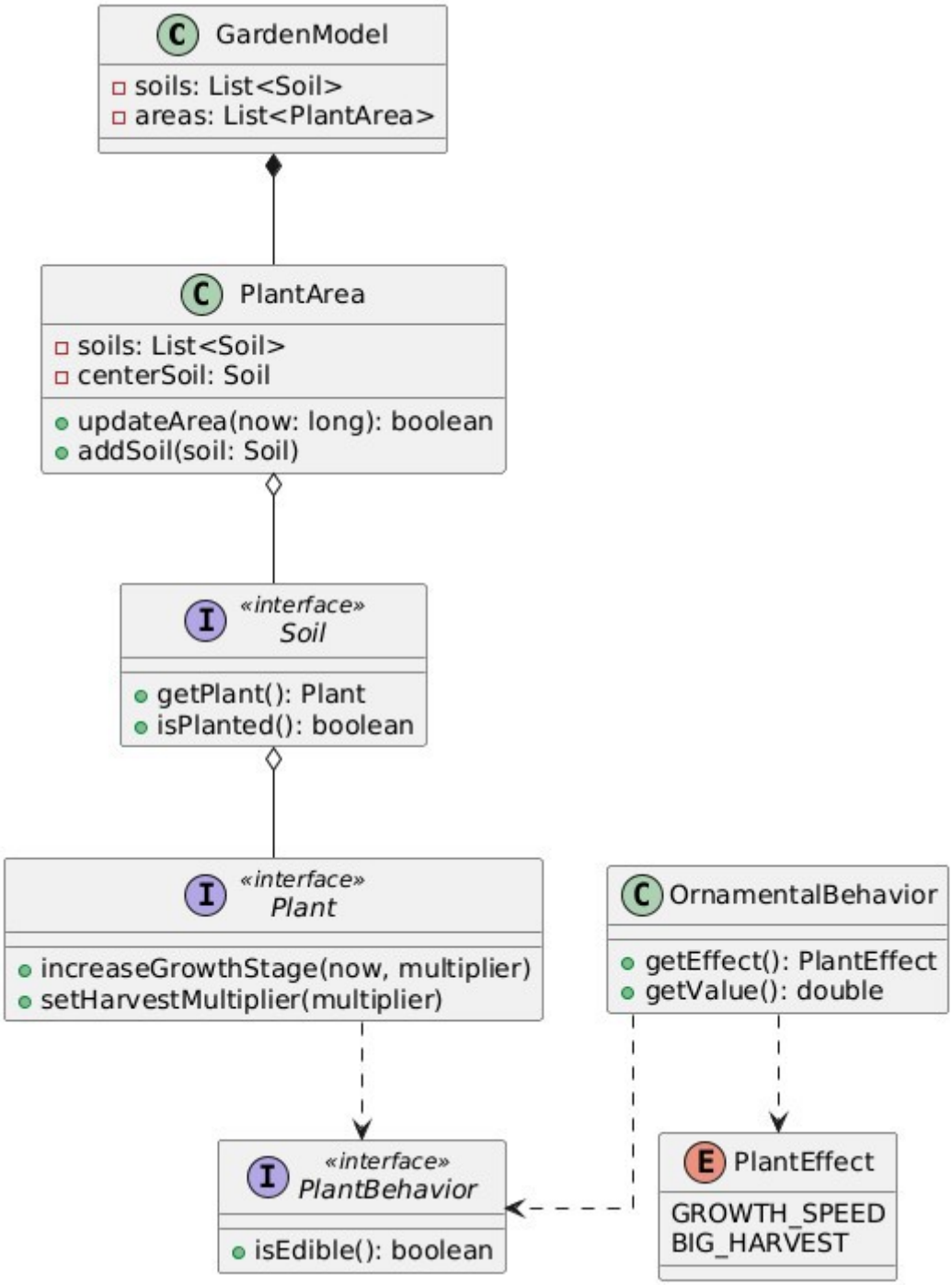

Figura 6: schema UML sull'applicazione dei bonus passivi delle piante ornamentali.

Selezione pianta da coltivare.

1. Problema

Il giocatore deve poter scegliere quale seme piantare tra le numerose specie scoperte senza appesantire l'interfaccia principale di gioco. È necessaria un'interfaccia che permetta di navigare tra le categorie, distinguere visivamente le piante già sbloccate da quelle ancora ignote e comunicare la scelta effettuata al sistema di semina nel mondo di gioco.

2. Soluzione

È stato implementato un sistema di selezione basato su un Controller dedicato e una visuale semplice e simile all'enciclopedia. La vista offre due modalità (Edibile e Ornamentale), permettendo di passare da una all'altra in base alla richiesta.

La comunicazione tra il menu di selezione e la logica di gioco avviene tramite l'interfaccia funzionale SeedSelectionListener. Questo permette al controller del gioco di conoscere la scelta dell'utente senza che il menu debba conoscere i dettagli del sistema di semina.

L'apertura del menu è legata al comando SELECT_SEED, che interrompe temporaneamente l'interazione col mondo per focalizzarsi sulla scelta della pianta, sfruttando il Comand Pattern per l'attivazione della finestra e il Observer Pattern la comunicazione del semento scelto.

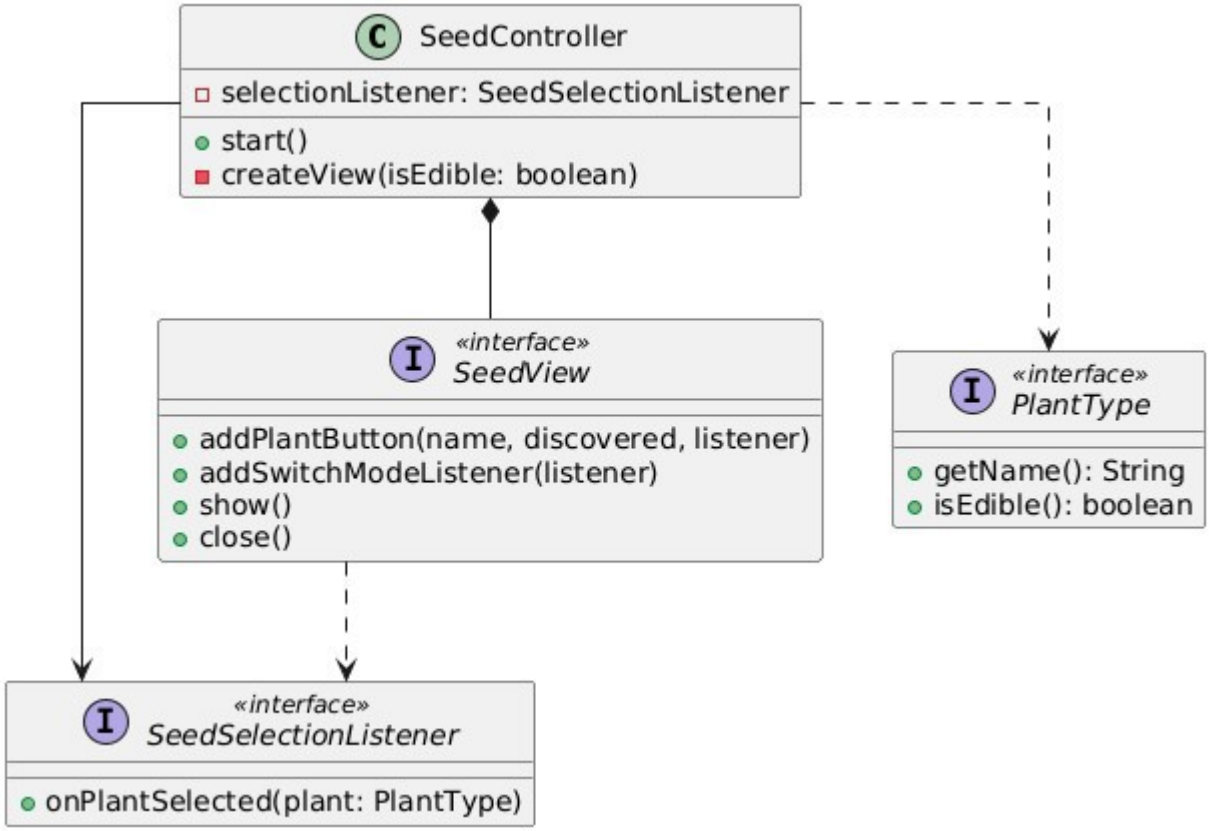

Figura 7: schema UML sulla selezione delle piante da coltivare.

2.2.2 Diego Bruschi

Lettura e scrittura di dati da file

- **Problema:** il gioco, ed in particolare la classe adibita alla creazione della mappa logica (TileMap), necessitano di avere accesso ad un file in cui vengano salvati gli oggetti SoilImpl, che a loro volta contengono oggetti PlantImpl che conservano tutte le informazioni relative alla singola pianta (stage corrente, bisogno di acqua, ecc…).

- **Soluzione:** il file di salvataggio viene generato e gestito dal file SoilSaving, il quale possiede i metodi loadGame(), saveGame() e reset() che vengono chiamati in altre parti del codice.
  In particolare:
- loadGame() utilizza un file (plant) che si trova sempre nella stessa directory (/plantsfarm), controlla se il file esiste, ed in caso affermativo si mette a controllare se gli oggetti salvati sono effettivamente SoilImpl; se lo sono li salva in una lista. Il metodo viene utilizzato all'interno di TileMap al momento della creazione della mappa logica per riempire i SoilImpl già creati in precedenza da un altro metodo.
- saveGame() prende in input una lista di SoilImpl che va poi a salvare direttamente sul file "plant". Il metodo non può venir chiamato costantemente all'interno del loop di gioco, in quanto al momento della chiusura di quest'ultimo il file rimarrebbe troncato e quindi illeggibile. Perciò il salvataggio avviene all'interno di ImplControllerGamePanel ogniqualvolta l'utente esegue un'azione tra piantare/raccogliere, innaffiare o rimuovere una pianta.
- reset() controlla se esiste un file di salvataggio ed in caso affermativo lo elimina. Il metodo viene chiamato all'interno di GameState ogniqualvolta viene premuto in gioco il bottone adibito al reset.

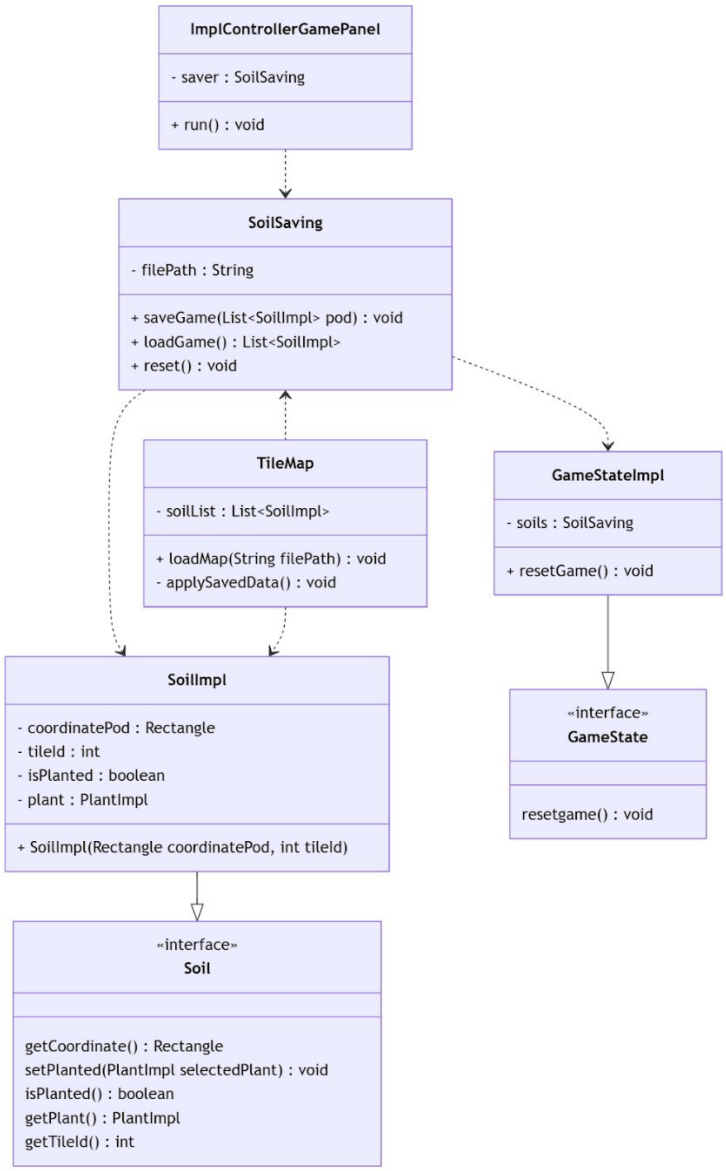

Figura 8: schema UML sulla gestione dei file.

Creazione della mappa logica

- **Problema:** il gioco ha bisogno di sapere da cosa la mappa sia effettivamente composta, in particolare di quali tiles siano quelli adibiti alla coltivazione e quali tiles siano invece oggetti solidi (muri, alberi, ecc..) e quindi non oltrepassabili dal giocatore.
- **Soluzione:** quando il gioco viene avviato, all'interno dell'ImplControllerGamePanel viene generata una nuova TileMap nella quale viene subito chiamato il metodo loadMap; quest'ultimo prende come input il file della mappa (map.txt) e legge gli indici dei singoli tiles in modo da riconoscere quali di questi sono SolidBlocks e quali ImplSoil, in modo da salvarli in apposite liste utilizzate da altre classi.

  Per quanto riguarda le collisioni la classe chiamata sempre da ImplControllerGamePanel è CollisionDetector, la quale calcola la posizione futura della hitbox del player e la confronta con quella dei SolidBlocks e, in caso di intersezione con uno di questi, impedisce al player di muoversi.

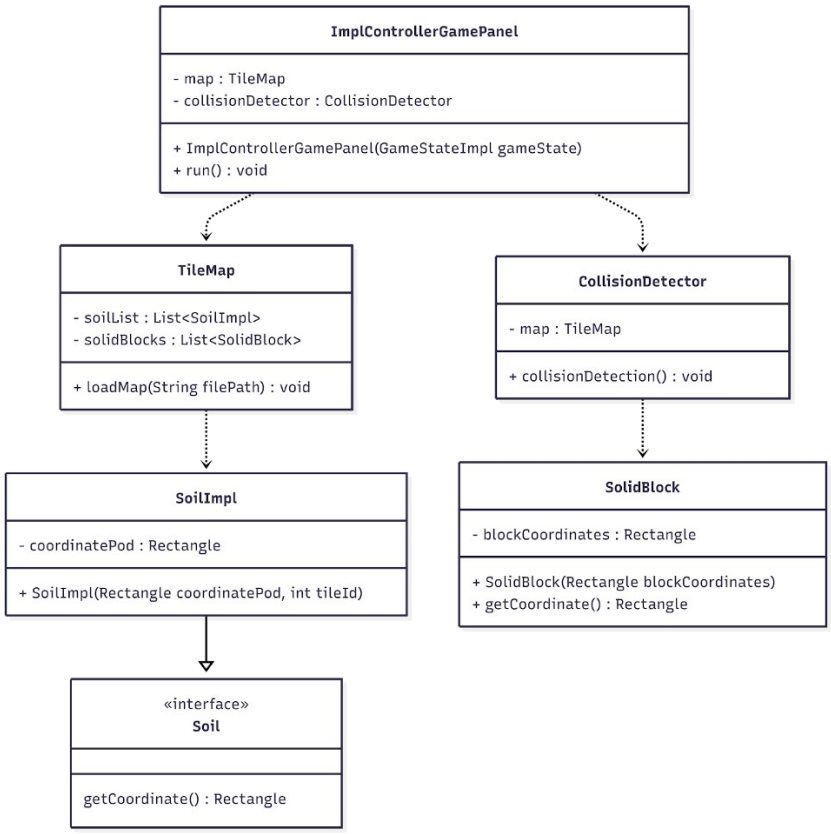

Figura 9: schema UML sulla creazione della mappa.

Gestione delle piante e dei buff all'interno della mappa

- **Problema:** è necessario un sistema di interazione tra il player e le singole piante per poter effettivamente giocare, e lo stesso vale per i buff che vengono generati nella mappa, i quali hanno bisogno di poter essere raccolti.

- **Soluzione:** L'architettura del sistema segue rigorosamente il pattern Model-View-Controller (MVC). La gestione della logica di dominio (dati delle piante, aree di coltura) è affidata al Model, mentre la "meccanica" e il coordinamento delle interazioni sono gestiti dai Controller.
La gestione dei Soil è in mano ad un GardenController, il quale si occupa di fornire i metodi necessari per interagire con le piante: piantare la pianta, innaffiarla e raccoglierne i frutti.
Ovviamente il tutto si appoggia alla classe PlantImpl derivata dall'interfaccia Plant, in cui si trovano salvati tutti i dati relativi a quella specifica coltura in quanto oggetto creato, ovvero tempo di crescita, cooldown per essere innaffiata, stadio di crescita corrente, ecc…
Le piante sono divise in due tipi: edibili ed ornamentali, e vedendo la mappa si realizza subito che ci sono aree separate al centro delle quali si può piantare solo una pianta ornamentale, ed intorno ad essa solo piante edibili (8 piante) che gioveranno dell'effetto fornito dalla coltura centrale. Queste aree vengono create e aggiornate da PlantArea e gestite da GardenModel. Le informazioni generali della pianta, quelle comuni a tutte le piante dello stesso tipo, vengono definite in PlantType.
Il GardenController viene controllato all'interno dell'ImplControllerGamePanel, il quale gestisce anche un altro controller, SpawningBuffsController, il quale si occupa della generazione di buffs appoggiandosi alla classe Buff e del loro spawn casuale sulla mappa in posizioni che non siano soils e blocchi solidi.

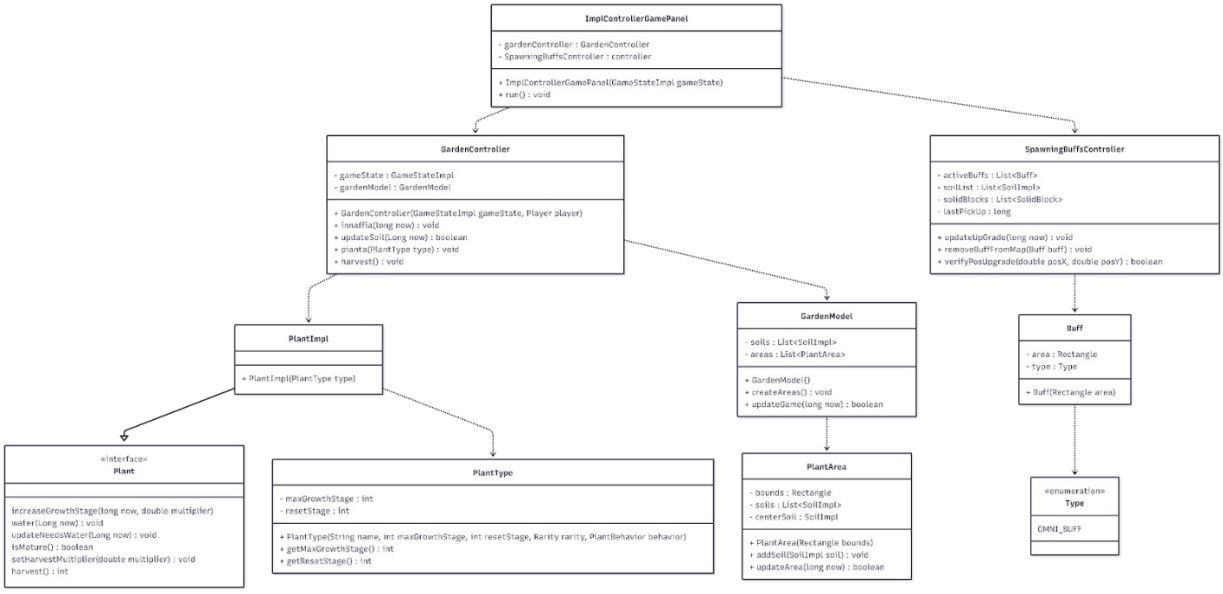

Figura 10: schema UML sulla gestione degli elementi nella mappa

Render della mappa, delle piante e dei buffs

- **Problema:** per visualizzare l'area di gioco si necessita un modo per dare un senso ai numeri presenti dentro al file .txt della mappa, in modo che ognuno si riferisca ad una specifica texture.
- **Soluzione:** i singoli indici presenti all'interno del file vengono assegnati ai Tile, una classe che utilizza BufferedImage per salvare la texture. I Tile vengono creati e salvati in un'altra classe, TileManager, che usando il file map.txt assegna ad ogni Tile la texture corrispondente e lo salva all'interno di un array di Tile. Per il caricamento dell'immagine tramite path viene utilizzata la classe SpriteLoader che accetta come input il path di un file e lo salva dentro ad una variabile che verrà assegnata al Tile in TileManager. Per visualizzare effettivamente i Tile si passa dall'ImplViewGamePanel, il quale tramite un metodo di TileManager disegna le Tile visibili dal player in quel momento, per risparmiare risorse. All'interno del ImplViewGamePanel vengono anche disegnate le effettive colture piantate; in questo caso le texture sono variabili, in base al tipo della pianta ed al proprio stadio di crescita corrente. Per reperire queste informazioni ci si appoggia a TileMap che fornisce la lista dei Soils, e alla classe Texture che in base al tipo di pianta ed al suo stage passati in input restituisce la texture corretta, che viene poi disegnata insieme ai Tile. Infine, tramite la classe SpawningBuffsController si ottiene la lista delle posizioni dei buffs attualmente attivi, i quali vengono poi disegnati insieme ai Tiles ed alle piante in ImplViewGamePanel.

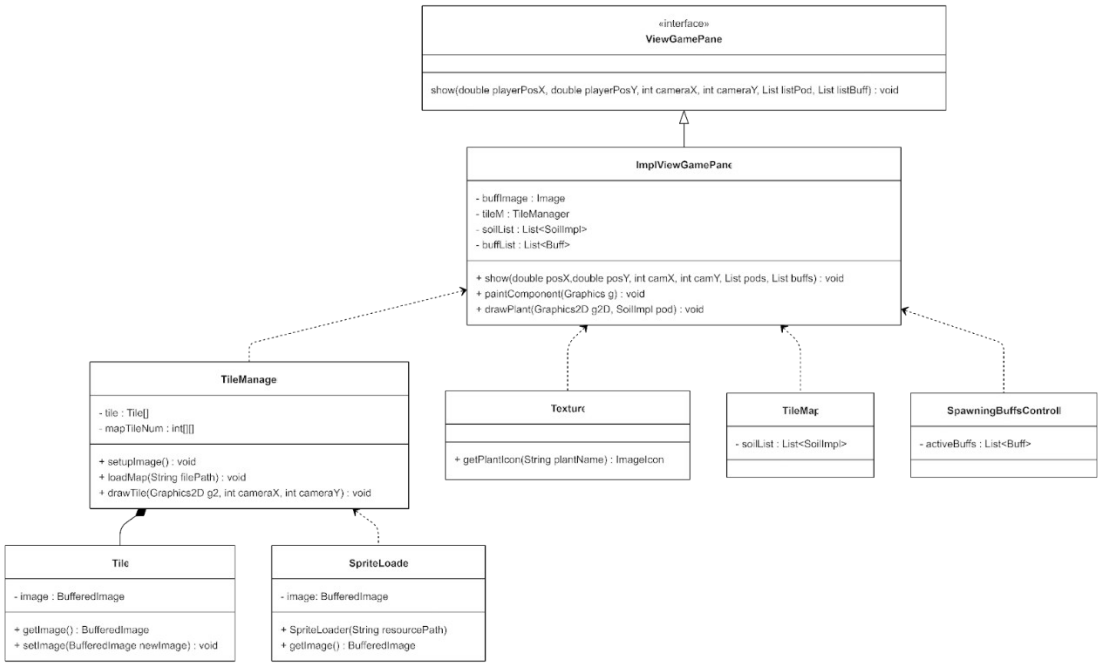

Figura 11: schema UML sul render della mappa.

2.2.3 Francesco Fusillo

Gestione del loop di Gioco

Problema : Le entità di gioco devono essere periodicamente aggiornate per rendere effettive le modifiche al loro stato, come il movimento, le azioni e le interazioni con l'ambiente

Soluzione: Il pattern utilizzato è l'MVC. La ViewGamePanel rappresenta la schermata principale di gioco che riceve gli input provenienti dall'utente. Il ControllerGamePanel aggiorna quindi la vista tramite il metodo show() della ViewGamePanel, al quale vengono passati i dati necessari alla rappresentazione dello stato corrente del gioco; all'interno di tale metodo viene inoltre invocato repaint() per aggiornare graficamente la scena. Il loop è gestito grazie alla classe Thread estesa dal ImplControllerGamePanel. A ogni iterazione il controllerGamePanel aggiorna tutti i model grazie ad altri controller.

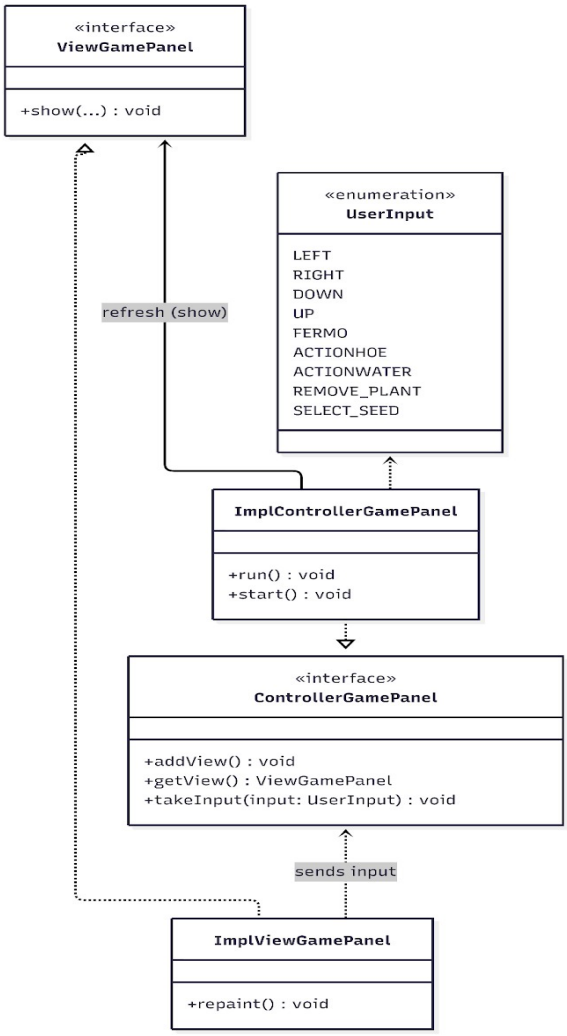

Figura 12 rappresentazione UML delle classi coinvolte per creare il Game Loop

Gestione Animazione
Problema: aggiornare il corrispettivo frame del player in base alle azioni che sta compiendo così da rendere le azioni del player uniche e dinamiche.

Soluzione: Per la gestione delle animazioni si è adottato il Pattern MVC. Questo Pattern è stato scelto per la alta responsività. Il SelectorFrames svolge il ruolo centrale nella gestione delle animazioni: riceve gli input provenienti dal ControllerGamePanel, e seleziona il frame corretto che la ViewGamePanel dovrà disegnare. Il problema principale in questa parte del progetto è stato coordinare correttamente l'azione del player con la durata dell'animazione. In particolare, al rilascio di un tasto il Player passa automaticamente allo stato di FERMO, interrompendo di fatto l'animazione dell'azione appena eseguita e impedendo il completamento naturale. Questo comportamento genera un'incoerenza tra la logica di gioco e la rappresentazione grafica, poiché alcune azioni risultano visivamente troncate prima della loro conclusione. Per risolvere il problema sono stati introdotti due tipi distinti di animazioni: AnimationCorsa e AnimationAzione. Le due implementazioni condividono una struttura simile dal punto di vista tecnico, ma differiscono nel comportamento. AnimationCorsa è pensata per gestire animazioni cicliche e continue, come il movimento del personaggio, garantendo fluidità attraverso un loop dei frame che rimane attivo

finché persiste lo stato di movimento. AnimationAzione, invece, è progettata come animazione “one-shot”: una volta avviata, non può essere interrotta anche se il tasto viene rilasciato, assicurando così il completamento naturale dell’azione e mantenendo coerenza tra input del giocatore, logica interna e resa grafica.

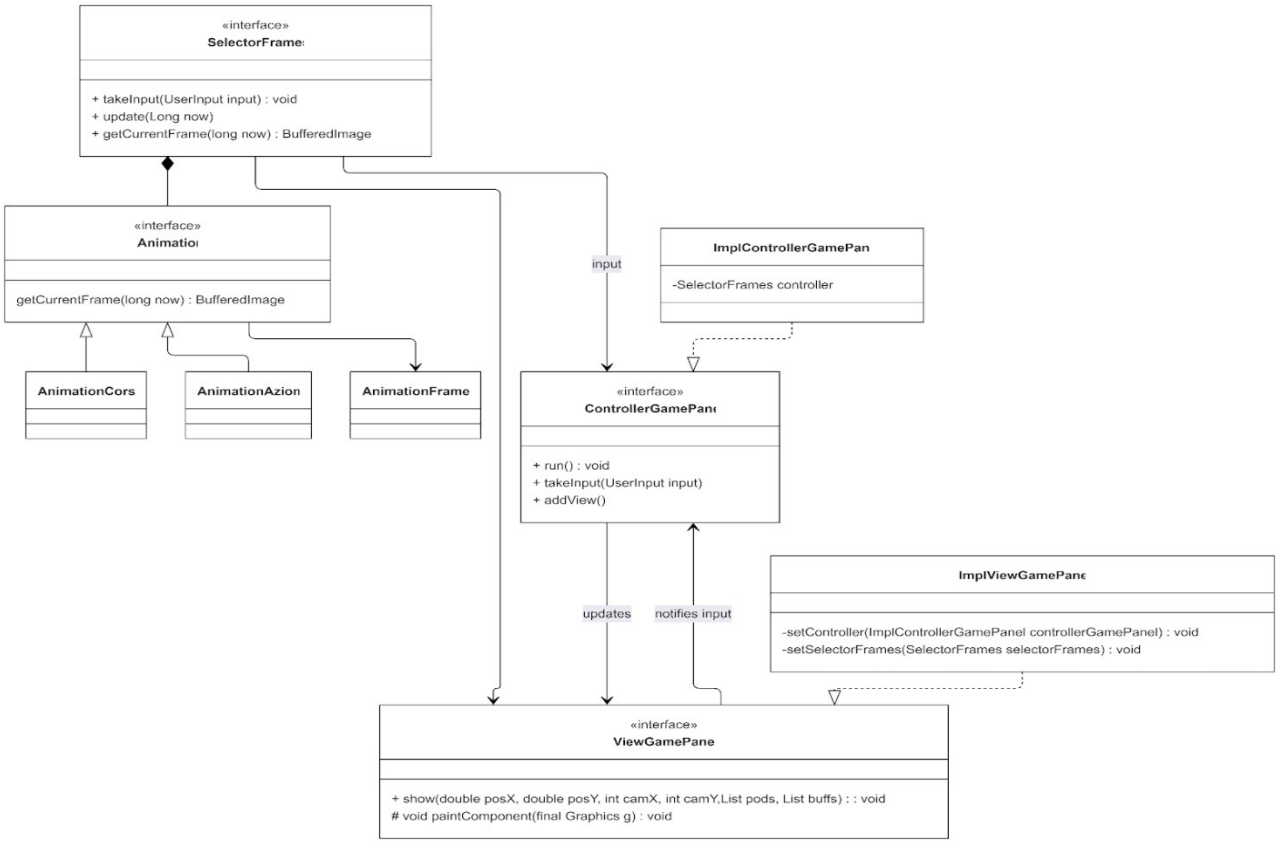

Figura 13 rappresentazione UML delle classi e interfacce coinvolte per le animazioni del Player

Creazione del Modello Player

Problema: La realizzazione dell’entità Player rappresenta il personaggio giocabile e include sia la gestione dei movimenti sia la gestione dei contenuti associati (ad esempio l’inventario che permette di utilizzare tutti gli oggetti offerti).

Soluzione: La creazione del Player è definita dal contratto offerto dall’interfaccia Player, che espone le funzionalità di base necessarie al personaggio, come il movimento e le azioni principali. Tale interfaccia è implementata dalla classe ImplPlayer, che contiene il comportamento comune a tutti i personaggi; da essa derivano due implementazioni concrete selezionabili: FarmExpert e Farmer.Per l’istanziazione del Player è stato adottato il design pattern Factory. La classe FactoryPlayer, ricevuto in input un valore di tipo TypePlayer, si occupa di creare l’istanza del personaggio corrispondente e di inizializzarne automaticamente l’inventario (inventario generato dalla FactoryInventario) in base alla scelta effettuata. La selezione del tipo di player avviene tramite la view SelectorPlayer, che invia l’input al ControllerGamePanel. Quest’ultimo delega quindi alla FactoryPlayer la creazione

dell'oggetto, mantenendo separata la logica di creazione dalla logica di utilizzo e riducendo l'accoppiamento tra le componenti del sistema.

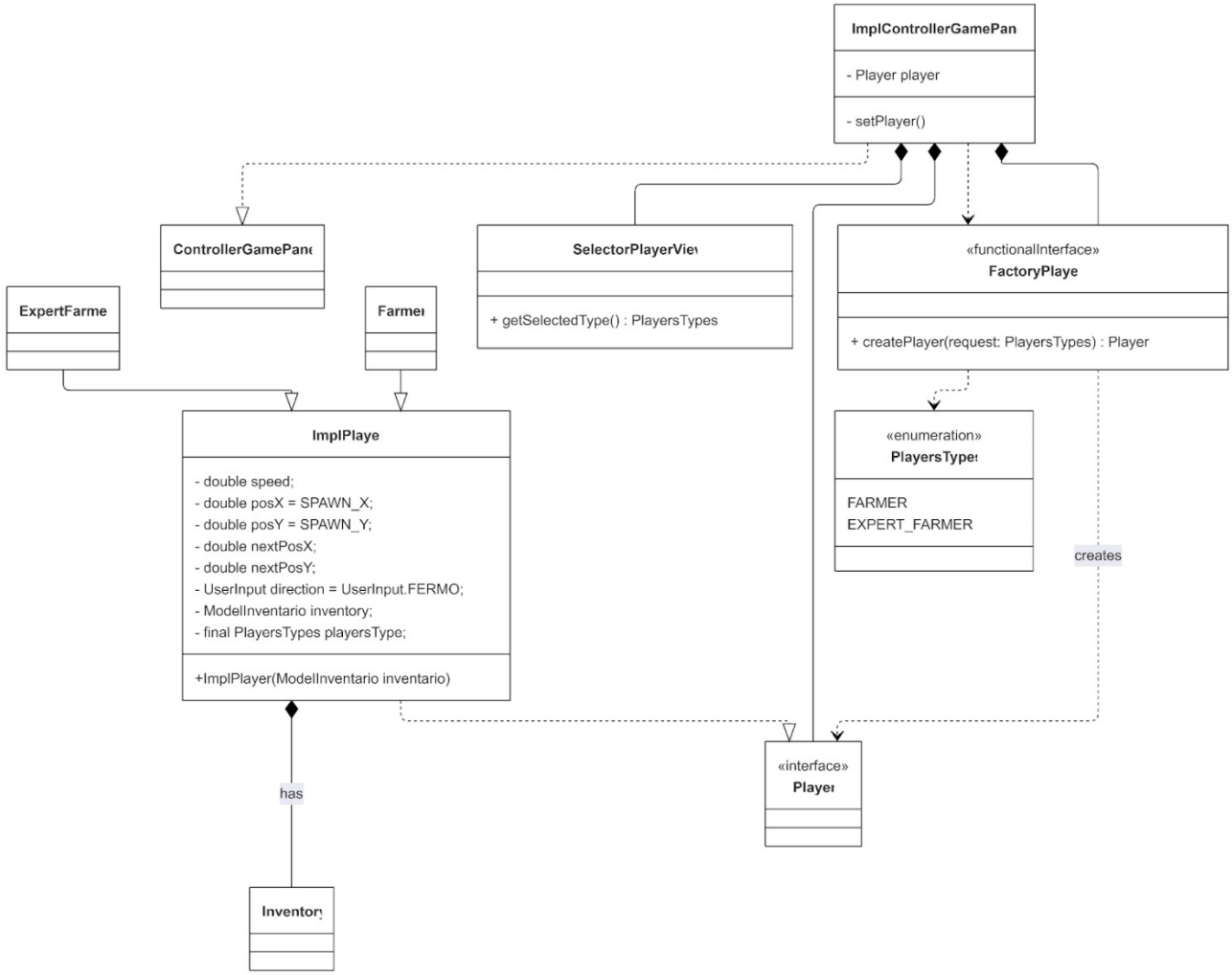

Figura 14 rappresentazione UML del modello Player e delle interfacce e classi che coinvolgono la sua creazione.

Modello Inventario

Problema:
Problema: Creare un modello di Inventario che venga inizializzato in modo diverso a seconda della tipologia di Player scelta, fornendo automaticamente gli strumenti coerenti con il personaggio selezionato.

Soluzione: Per risolvere questo problema è stato adottato il Factory pattern. Questa scelta strutturale risulta particolarmente adeguata quando è necessario creare oggetti con configurazioni differenti in base a un parametro esterno, in questo caso il tipo di Player. Attraverso la Factory, il sistema delega la responsabilità della creazione dell'Inventario a un componente dedicato (FactoryInventario), che istanzia il modello con gli strumenti appropriati (ad esempio strumenti base per un player standard oppure strumenti già potenziati per una versione avanzata del personaggio). In questo modo si evita di inserire logiche condizionali complesse all'interno del Player e si mantiene una chiara separazione

delle responsabilità. Il model Inventory, oltre alla fase di creazione, espone anche un insieme di metodi per accedere agli strumenti disponibili, verificarne lo stato (come livello o rarità) e gestirne l'utilizzo durante il gioco. Questo approccio migliora la modularità del codice, facilita l'estensione futura con nuovi tipi di Player o nuove configurazioni di Inventario e mantiene il sistema coerente con i principi di progettazione orientata agli oggetti.

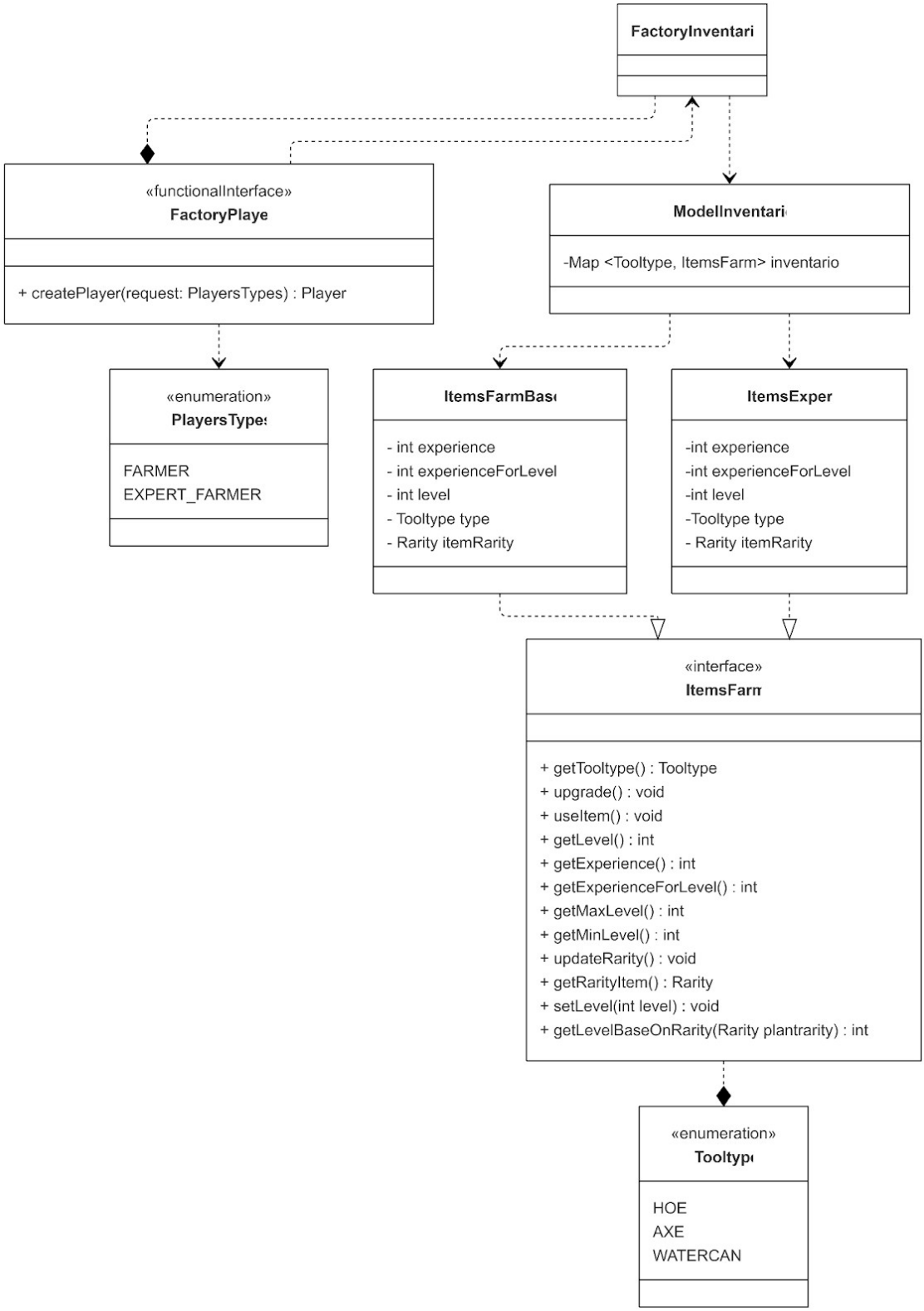

Figura 15 rappresentazione UML del modello Inventario

Creazione View dell'inventario

Problema: creare una GUI per mostrare gli item del giocatore e per ognuno i rispettivi progressi.

Soluzione: La View dell'inventario è stata progettata come una composizione di pannelli, adottando il pattern Composite per gestire la View come un insieme di componenti grafici indipendenti ma coordinati tra loro. Ogni pannello estende la classe JPanel e implementa l'interfaccia UpdatablePanel, che espone il metodo update() come funzione principale per garantire la corretta sincronizzazione grafica con il modello. In questo modo, quando lo stato dell'inventario cambia (ad esempio dopo un'azione o un upgrade), ogni sezione dell'interfaccia può aggiornarsi in modo immediato e coerente. Questa struttura rende i pannelli altamente intercambiabili: ogni sezione può essere sostituita con un'implementazione diversa senza dover riscrivere l'intera View. Di conseguenza, il sistema può essere facilmente esteso introducendo nuovi layout o nuove sezioni (ad esempio per differenti tipologie di player o nuove categorie di item), mantenendo invariata la struttura generale della View e limitando le modifiche alle sole componenti interessate.

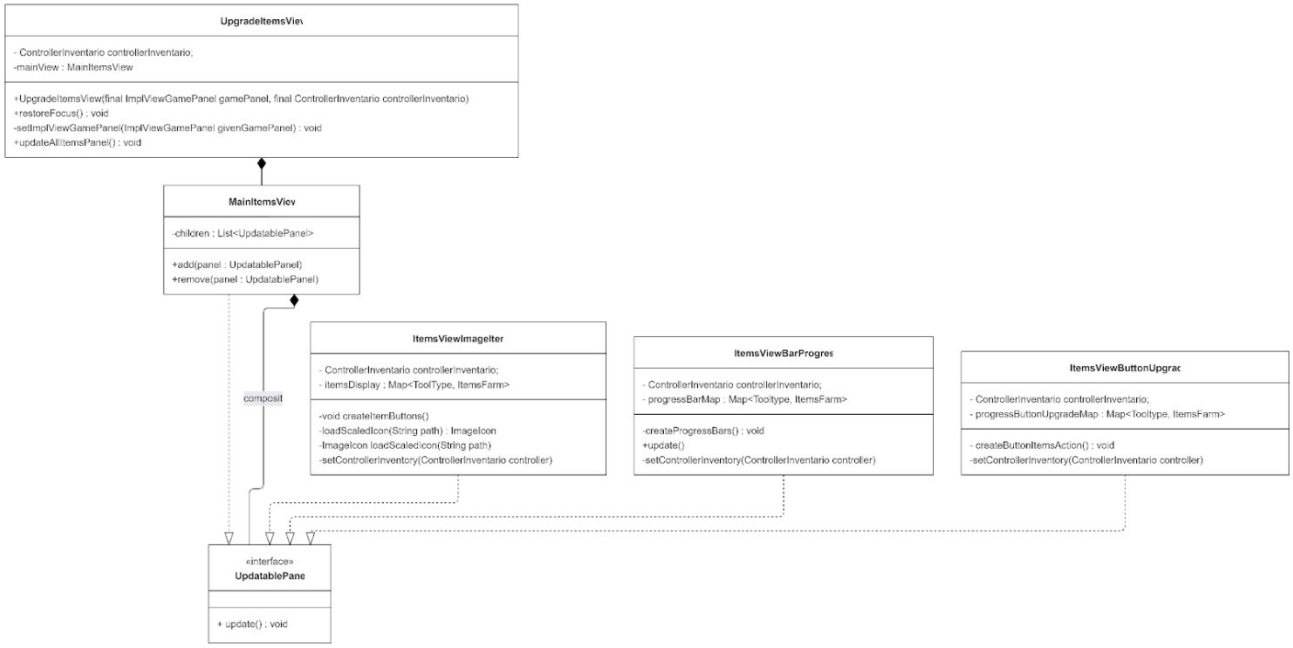

Figura 16 rappresentazione Uml della View Inventario

Logica dell'Inventario

Problema: Gestire correttamente la comunicazione tra la View dell'inventario e il modello, garantendo che gli input provenienti dall'interfaccia grafica vengano tradotti in aggiornamenti coerenti del model Inventory e che ogni modifica dello stato del modello venga immediatamente riflessa nella GUI, mantenendo sincronizzazione e coerenza tra logica applicativa e rappresentazione

grafica.

Per raggiungere questo obiettivo è stato adottato il pattern MVC. Il Model è rappresentato dall'inventario contenuto all'interno del Player, che gestisce dati come item posseduti, livelli, esperienza e rarità. Il Controller dell'inventario ha il compito di mediare tra View e Model: intercetta gli eventi generati dalla GUI (ad esempio richieste di upgrade), invoca le operazioni appropriate sul modello e successivamente aggiorna la vista. La View, invece, si occupa esclusivamente della rappresentazione grafica dello stato corrente dell'inventario, senza contenere logica decisionale. La scelta dell'architettura MVC deriva dall'elevata responsività e dalla chiara separazione delle responsabilità che essa offre. Per aggiornare i dati correttamente non viene fornito il riferimento diretto all'istanza originale dell'inventario, ma una copia del suo stato. In questo modo si evita che la componente grafica possa modificare direttamente il modello, preservando l'integrità dei dati e rispettando il principio di incapsulamento. Eventuali modifiche vengono sempre effettuate tramite il Controller, che rimane l'unico responsabile dell'accesso in scrittura al Model.

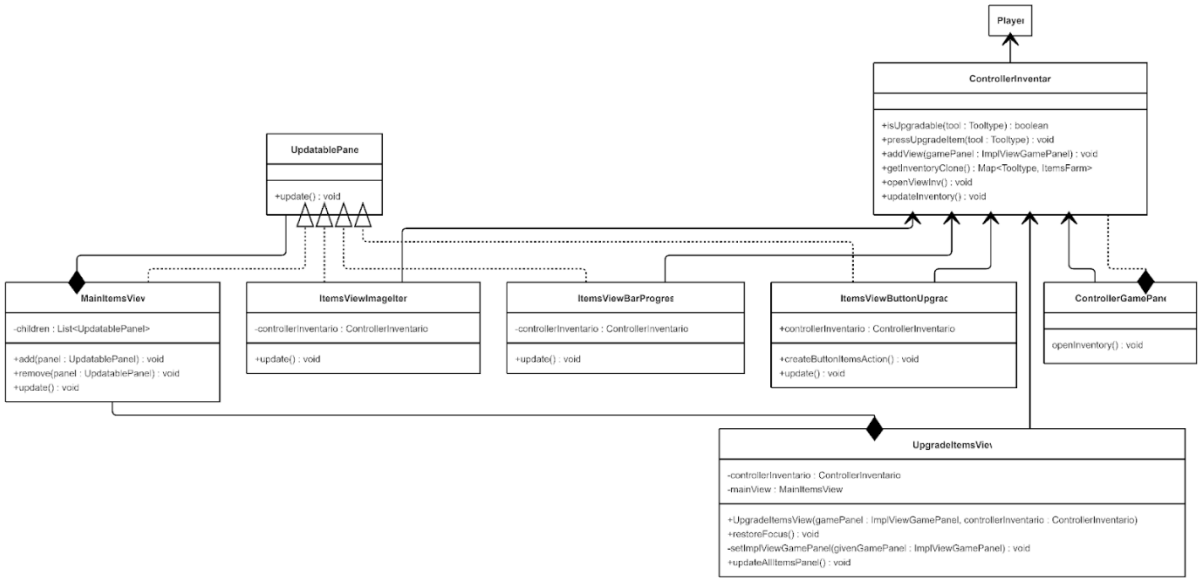

Figura 17 rappresentazione UML Logica dell'inventario

Gestire azioni e movimenti del Player

Problema: gestire le azioni/movimenti del Player e aggiornare in maniera corretta il model Player.

Soluzione: Il problema affrontato in questa sezione consiste nel gestire movimenti e azioni del Player, assicurando che ogni input si traduca in un aggiornamento coerente dello stato del modello, sia per il Player sia per l'Inventario contenuto al suo interno. La soluzione adottata affida questa responsabilità all'ActionHandler che, pur non essendo il controller principale, svolge un ruolo analogo a un controller: riceve gli input inoltrati dal ControllerGamePanel e applica la logica necessaria per aggiornare il modello del Player (posizione, stato, azioni) e il modello dell'Inventario (ad esempio verifiche di utilizzo o modifiche conseguenti alle azioni). Per garantire correttezza e coerenza con lo stato della mappa e delle colture, l'ActionHandler opera in coordinazione con il ControllerGarden e il ControllerSeed: il primo fornisce le informazioni legate alle posizioni e allo stato dei soil, mentre il secondo permette di ottenere i dati relativi alla pianta o al seme selezionato, rendendo possibile validare l'azione richiesta e applicarla nel contesto corretto.

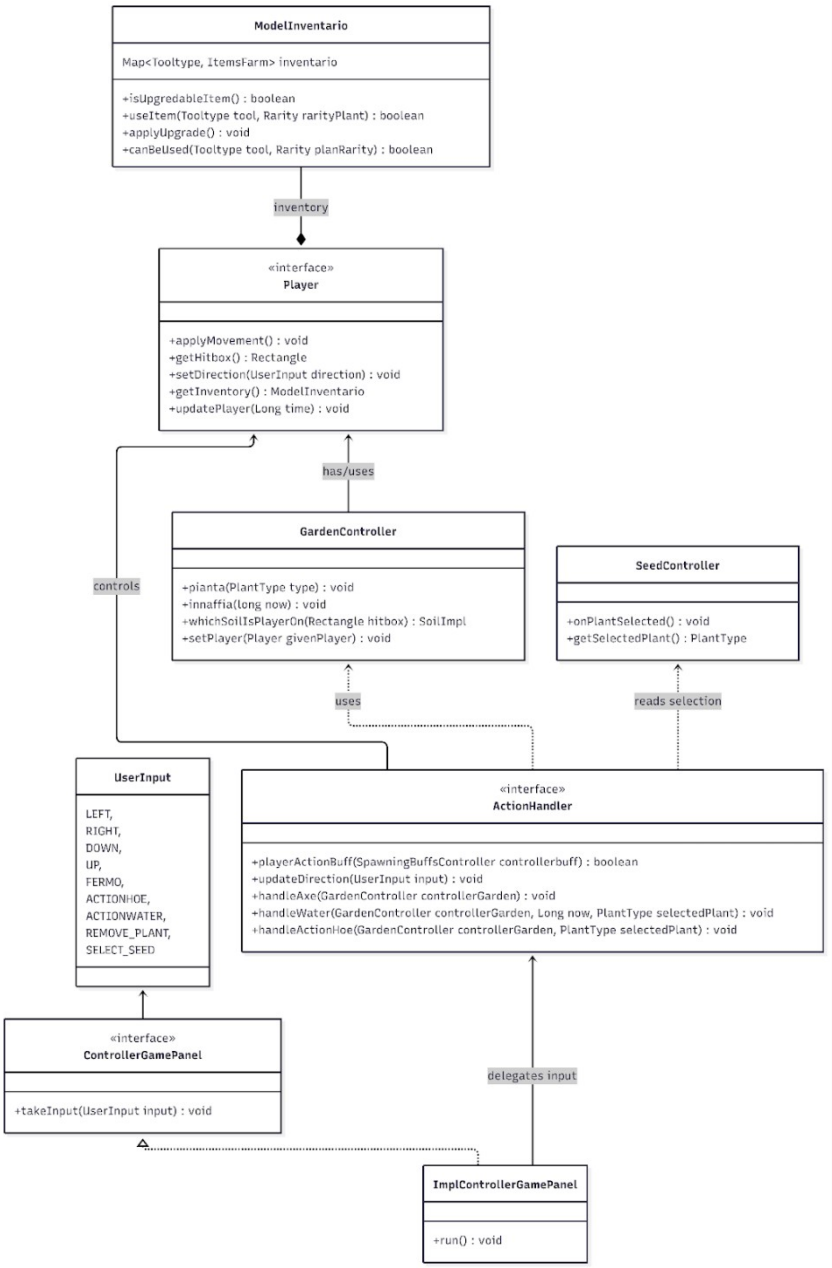

Figura 18 rappresentazione UML della gestione azioni/movimento del Player

Modello Camera

Problema: Spostare lo schermo in base allo spostamento del player. Rendering di tutti i tile e oggetti della mappa in base alla posizione dello schermo e alle loro posizioni del mondo.

Soluzione: L'architettura utilizzata nel progetto è il pattern Model-View-Controller (MVC), che consente di separare in modo chiaro le responsabilità tra gestione dei dati, logica di controllo e rendering grafico. In questo contesto, la classe Camera rappresenta il modello della telecamera. Essa contiene tutte le informazioni necessarie per determinare quale porzione della mappa debba essere visualizzata, come la posizione corrente della camera e le dimensioni della viewport. La telecamera

non si occupa né di disegnare né di leggere input: mantiene esclusivamente il proprio stato. L'aggiornamento della telecamera avviene esclusivamente all'interno del controller ImplControllerGamePanel. Durante il game loop, il controller richiama il metodo followPlayer(), che aggiorna la posizione della camera in base alla posizione attuale del giocatore. In questo modo viene rispettata la separazione delle responsabilità: il modello conserva lo stato, il controller lo modifica, mentre la view si limita a utilizzarlo per il rendering. Le coordinate della telecamera sono utilizzate poi come punto di riferimento per disegnare i vari tile/piante in base alla posizione del Player nel mondo cosi da poter garantire la corretta posizione degli oggetti nel mondo.

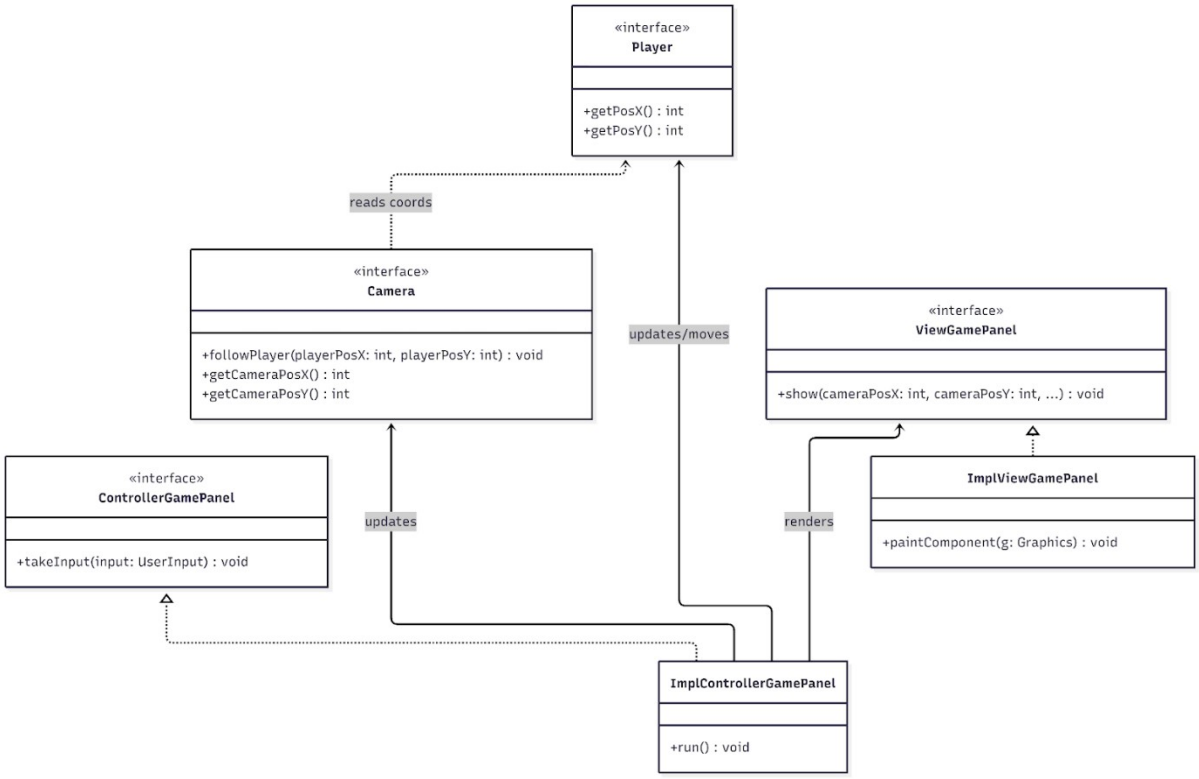

Figura 19 modello Camera

Problema: creare un sistema di salvataggio capace di immagazzinare le informazioni sui livelli degli item. Allo stesso tempo il sistema deve essere in grado di caricare i dati salvati nei files.

Soluzione: Per risolvere il problema è stato introdotto un livello di astrazione tramite l'interfaccia Memory, che definisce le operazioni generiche di salvataggio e caricamento dei dati. In questo modo il modello e i controller non dipendono direttamente dal filesystem ma comunicano attraverso un contratto astratto. La classe SavePlayer si occupa esclusivamente della serializzazione e deserializzazione dell'inventario, convertendo gli oggetti di gioco in una rappresentazione testuale e viceversa. Essa non conosce il percorso reale del file ma utilizza l'interfaccia Memory, rendendo il sistema facilmente estendibile. Il controller ManagerSavingPlayer coordina invece l'operazione e decide quando effettuare il salvataggio o il caricamento in base al tipo di player. Il salvataggio viene fatto ogni volta che viene compiuta un'azione o viene aperta la View dell'inventario. Grazie a questa struttura la logica di gioco rimane separata dalla persistenza, il modello resta indipendente dal supporto fisico di memoria ed è possibile sostituire il sistema di salvataggio senza modificare il resto

dell'architettura. Il salvataggio è stato limitato soltanto ai livelli così da evitare che il Player debba sempre ricominciare dall'inizio. Il punto debole di questo sistema ricade nella scelta del Player, se esso dovesse dimenticarsi di upgradare gli items, l'esperienza accumulata in eccesso verrebbe persa.

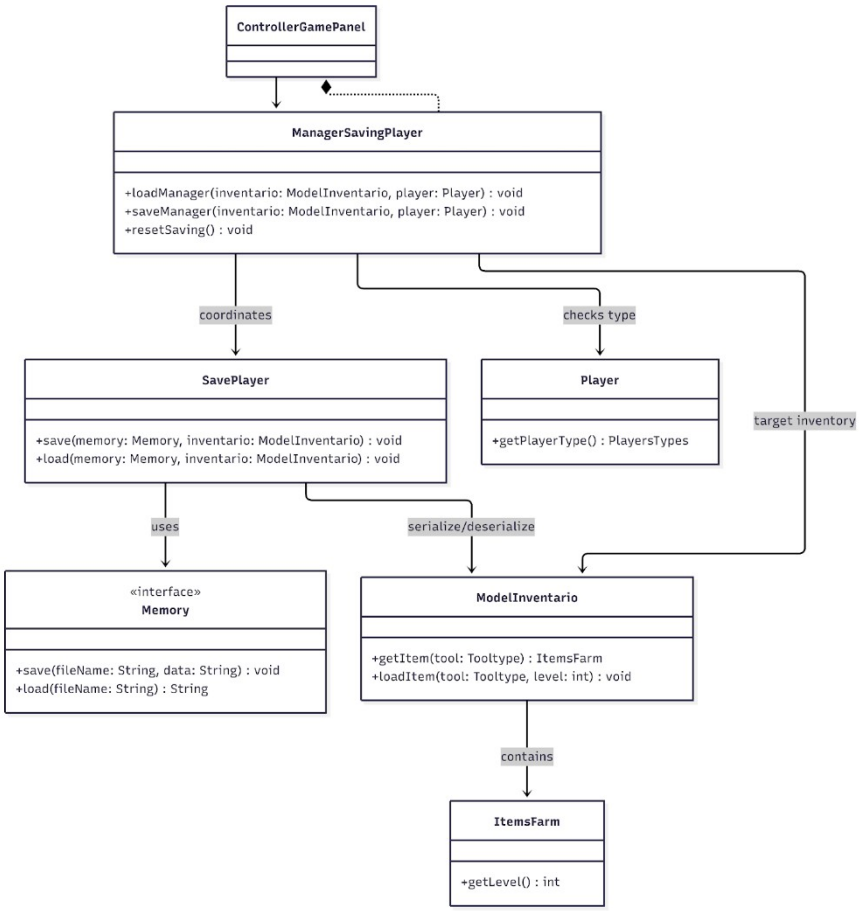

Figura 20 rappresentazione salvataggio Player

Sviluppo

3.1 Testing automatizzato

3.1.1 Enrico Michelini

Ho focalizzato le attività di testing sul modello dei dati, garantendo la coerenza dello stato di gioco dopo ogni transizione e la corretta persistenza su file:

- Gestione del Magazzino (Storage):

  Verificare l'inizializzazione degli slot per le sole piante edibili, controllare il valore degli slot dopo la rimozione e l'aggiunta di raccolti e integrazione con la persistenza di file.

- Logica del Negozio (Shop):

Verificare che vengano generate richieste coerenti con le piante sbloccate nell'enciclopedia, controllare il valore dopo le transizioni dei modelli che ne vengono modificati, ovvero il portafoglio e il magazzino.

Verifica dell'effettivo funzionamento dell'acquisto di "MysteryBox" per lo sblocco di nuove piante.

- Enciclopedia delle piante (Encyclopedia):

  Verifica dell'inserimento delle piante, il loro effettivo sblocco, e le modalità di filtraggio in base alle caratteristiche.

3.1.2 Diego Bruschi

Per quanto riguarda la mia parte del progetto ho testato le sezioni principali da me sviluppate. Nello specifico ho eseguito test su:

- Piante: creazione della pianta, con controllo della correttezza delle statistiche di partenza, innaffiatura della pianta considerando anche il cooldown del bisogno di acqua, crescita della pianta, testando anche i blocchi implementati (la pianta ha bisogno di acqua per crescere e devono essere passati 30 secondi dall'ultimo cambio di stage), e harvest della pianta, con controllo dell'effettiva produzione di frutti da vendere.
- Mappa logica: creazione della mappa logica con controllo del conteggio di tile considerati come soils coltivabili e tile considerati solidi e quindi invalicabili dal player, test della correttezza dell'assegnazione della posizione dei solid blocks e dei soils, sia per piante ornamentali che edibili.
- Buff: creazione di un buff in una posizione prestabilita e controllo del tipo, controllo della illegal position e della legal position di buff creati ad hoc per il test.

3.1.3 Francesco Fusillo

Per garantire il corretto funzionamento delle componenti principali del sistema, sono state eseguite diverse attività di testing mirate alle parti fondamentali dell'architettura. In particolare, sono state verificate le seguenti funzionalità:

- Creazione del Player e dei suoi diversi tipi: sono stati testati i metodi delle factory per assicurarsi che, a partire dal valore di PlayersTypes, venga istanziata correttamente la classe concreta (Farmer o ExpertFarmer), con le caratteristiche previste.
- Creazione dei diversi tipi di Inventory: sono stati verificati i metodi della FactoryInventory per controllare che l'inventario venga inizializzato correttamente in base al tipo di player selezionato, includendo gli item appropriati, i livelli iniziali e le eventuali differenze tra configurazione base ed esperta.
- Verifica dell'utilizzo degli item e dell'aumento dell'esperienza: sono stati effettuati test sul modello per controllare che l'uso di un item aggiorni correttamente lo stato interno (ad esempio consumo, applicazione effetto, incremento esperienza) e che il sistema di progressione rispetti le regole definite (livello, rarità, requisiti minimi, ecc.).

3.2 Note di sviluppo

3.2.1 Enrico Michelini

Utilizzo di Stream: al permalink:

https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/controller/menu/ShopController.java#L137

Espressioni Lambda: al permalink:

https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/controller/MainScreenController.java#L38

3.2.2 Diego Bruschi

Utilizzo di Input Stream per la lettura del file di salvataggio o per la lettura del file contenente la mappa ai link:

https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/model/garden/SoilSaving.java#L49-L50 https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/model/tiles/TileMap.java#L68-L69 https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/view/map/TileManager.java#L133-L134

Utilizzo di Output Stream per la scrittura/sovrascrittura del file di salvataggio al link: https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/model/garden/SoilSaving.java#L29-L30

3.2.3 Francesco Fusillo

Uso di Thread:

https://github.com/SignorDoDo22/OOP25-PlantsFarm/blob/19f1b807ba12d9cf308cea9379652222433790d4/src/main/java/it/unibo/plantsfarm/controller/gamepanel/ImplControllerGamePanel.java

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Enrico Michelini

Non è stato il mio primo approccio alla realizzazione di un videogioco, ma è stata la prima volta che ho lavorato in un progetto di gruppo supportato da una consapevolezza tecnica specifica. Questo mi permette di fare una valutazione critica dei miglioramenti e delle differenze sostanziali tra le due esperienze.

Sono pienamente soddisfatto del percorso svolto: il progetto mi ha fornito basi solide sulla stesura professionale del software. Rispetto alla mia esperienza precedente, caratterizzata dall'assenza di

un'architettura definita e da conoscenze preliminari scarse, ho riscontrato un miglioramento drastico nella qualità del codice e nell'organizzazione del lavoro.

Lavorare in gruppo non è stato facile, ma ho trovato una buona comunicazione con i membri del mio progetto, abbiamo diviso le parti in modo definito ma allo stesso tempo flessibile. Abbiamo adottato un approccio collaborativo: ogni modifica impattante sulle componenti altrui veniva discussa preventivamente, dimostrando disponibilità al dialogo specialmente nella definizione delle interfacce comuni. Nello specifico, essendomi occupato dell'interfaccia del Menù, il mio modulo presentava un basso accoppiamento con le logiche di gioco e movimento implementate dagli altri. Questo mi ha permesso di lavorare con maggiore autonomia.

A posteriori rivaluterei completamente la scelta del gioco, essendo un progetto universitario dove il risultato viene proposto in una piccola demo, avrei scelto un gioco più dinamico e veloce che avrebbe garantito un impatto più immediato, la scelta di un gioco gestionale risulta meno efficace in una presentazione limitata.

Nel complesso sono molto soddisfatto dell'esperienza e delle competenze maturate. Il mio obiettivo per i prossimi anni universitari è sviluppare una app o un software completo per arricchire il mio curriculum e grazie a questo corso e al progetto svolto, sento di averne acquisito le basi fondamentali.

4.1.2 Diego Bruschi

Arrivato alla fine del progetto mi reputo abbastanza soddisfatto del mio operato, per quanto, avendo avuto più tempo, avrei potuto fare di più, soprattutto nell'organizzazione dei file compresi nella mia parte, dei metodi al loro interno, e anche nello sviluppo di altre funzioni di gioco per rendere il tutto ancora più dinamico e coinvolgente. Nonostante ciò, ritengo che il mio lavoro sia stato abbastanza completo al fine di soddisfare i requisiti elencati nella proposta e che gli obiettivi che mi ero preposto sono stati raggiunti, favorendo il mio apprendimento dell'utilizzo di Java e anche la mia capacità di lavorare in team. Durante il progetto è capitato spesso infatti di dover lavorare a stretto contatto con i miei colleghi in quanto alcune delle sezioni del progetto a noi assegnate si intrecciavano tra loro, e questo mi ha insegnato a comunicare con altre persone in ambito "lavorativo" e ad adattare il mio lavoro in modo che sia conforme agli standard e che si incastri con il modo di lavorare dei colleghi.

Detto questo per il futuro io ed un altro componente del team abbiamo intenzione di continuare a migliorare l'applicazione per renderla completa, aggiungendo nuove dinamiche di gioco per renderlo più "realistico"; una volta soddisfatti del risultato pensavamo di caricarlo su piattaforme adibite ai giochi, quali Steam, Itch.io, ecc…

4.1.3 Francesco Fusillo

Alla fine di questo lungo percorso di sviluppo posso dirmi soddisfatto del lavoro svolto.

Sarebbe stato interessante introdurre ulteriori elementi di gameplay, ma il tempo a disposizione non lo ha permesso; nonostante ciò, ritengo che la struttura generale della mia parte di progetto sia solida e coerente.

Rimangono però alcuni aspetti migliorabili: in particolare, l'utilizzo dei design pattern non è stato sempre approfondito e questo limita in parte la possibilità di ottenere componenti più riutilizzabili e facilmente estendibili.

Posso comunque affermare che il mio lavoro sia stato sufficientemente completo da soddisfare i requisiti elencati nella proposta.

La comunicazione all'interno del gruppo e una buona organizzazione si sono rivelate aspetti fondamentali per la riuscita del progetto, soprattutto nella gestione delle parti condivise.

Ritengo inoltre che il mio gruppo sia stato molto collaborativo e serio nell'impegno.

Appendice A

Guida utente

All'avvio dell'applicazione verrà richiesto all'utente di scegliere tra due tipi di agricoltori, farmer o expert farmer, ognuno con le sue caratteristiche. Il Farmer offre un'esperienza del gioco classica, con un avanzamento più graduale dato dalla necessità di aumentare il livello dei propri utensili, ed una velocità di movimento ridotta, mentre l'expert farmer gioverà di utensili già al massimo livello ed una velocità di movimento superiore. Una volta fatta la scelta, il gioco si avvierà caricando la mappa ed eventuali dati salvati in memoria, che comprendono: progressione dello sblocco delle piante, elementi presenti nello storage, denaro accumulato, presenza e caratteristiche delle colture piantate sui soils (stage, necessità di acqua) e livelli degli utensili (se si usa un farmer normale).

Per iniziare a giocare il player dovrà muoversi sull'asse x e y usando i tasti WASD. Inizialmente l'utente avrà a sua disposizione 500 monete e come unica coltura disponibile la pianta di carote, che essendo di rarità comune può essere piantata, innaffiata, raccolta e rimossa con gli utensili del farmer al livello 1. I livelli degli strumenti sono disponibili in un menù consultabile premendo il tasto F, in cui sono presenti delle barre di progressione che mostrano quanto manca per raggiungere il prossimo livello ed un tasto upgrade per ogni utensile.

Per piantare la prima coltura è necessario posizionarsi sopra ad una zona coltivabile adibita alle piante edibili, ovvero una zona in cui la terra non sia contornata con dell'erba verde scuro (quelle al centro delle matrici 5x5), aprire con il tasto P la selezione della pianta da coltivare e infine premere il tasto Q. Una volta creata la pianta avrà subito bisogno di acqua per crescere (condizione necessaria per il cambio di stage è l'essere stata innaffiata almeno una volta nello stage corrente), e si potrà usare l'innaffiatoio premendo il tasto E. Quando la coltura raggiungerà lo stadio finale di crescita, segnalato da un punto esclamativo dorato, sarà possibile raccoglierne i frutti usando sempre il tasto Q. Nel caso in cui tutte le zone adibite alla coltivazione delle piante edibili fossero occupate, si ha comunque la possibilità di rimuoverne alcune tramite l'ascia, utilizzabile posizionandosi sulla pianta da rimuovere e premendo il tasto R. Dopodiché sarà possibile piantare una nuova pianta, la quale potrà essere scelta da un menù a comparsa attivabile con il tasto P.

La totalità delle colture raccolte è visibile in un menù attivabile premendo il secondo bottone nella colonna di destra con il simbolo del baule. I vegetali ottenuti possono essere poi venduti dentro il negozio, visitabile cliccando il primo tasto nella colonna di destra con il simbolo del negozio.

All’interno di questo menù, sulla sinistra, si trova una sezione in cui sono elencate le piante da vendere (tra quelle sbloccate), con la quantità richiesta ed il relativo guadagno. Con i propri guadagni sarà possibile comprare nuove colture attraverso la sezione di destra, in cui sono presenti quattro bottoni: uno per comprare una pianta casuale generica, uno per le piante edibili, uno per quelle ornamentali, ed uno per cambiare le piante richieste nella sezione di sinistra per la vendita.

Le piante sbloccate sono consultabili all’interno dell’enciclopedia, visualizzabile premendo il terzo bottone nella colonna di destra; ogni pianta è completa di descrizione personalizzata, rarità e immagini che la rappresentano in ogni suo stage.

Infine, nel quarto bottone in basso della colonna di sinistra, con il simbolo dell’ingranaggio, si trovano le impostazioni: all’interno di questo pannello si trovano vari bottoni, ovvero:

- Pulsante Resume per tornare al gioco;
- Pulsante Commands per visualizzare tutti i tasti utilizzabili per interagire con il gioco;
- Pulsante Reset Data per eliminare l’attuale salvataggio.
- Pulsante Credits per visualizzare le fonti che hanno fornito gli assets usati nell’applicazione.
- Pulsante Exit Game per chiudere il gioco.

Bibliografia

- music: https://downloads.khinsider.com/game-soundtracks/album/plants-vs.-zombies/07.%2520Zen%2520Garden.mp3
- Frame Player presi da: Cute Fantasy RPG - 16x16 top down pixel art asset pack by Kenmi
  Immagini elaborate con: Piskel - Free online Sprite editor
  https://www.piskelapp.com/
  https://kenmi-art.itch.io/cute-fantasy-rpg